

Framework Research on the Implementation of Automated Test User Requirements

Yiming Xiao
Hubei university of technology
Wuhan, China
+8615623306380
xym1164434191@live.com

Liang Xiao
Hubei university of technology
Wuhan, China
+8618062500600
48453626@qq.com

Zhu Li
Hubei university of technology
Wuhan, China
+8617798315233
lizhu2013@foxmail.com

ABSTRACT

MVC is a software design model invented by Xerox PARC for Smalltalk in the 1980s. It has been widely used in software design and development. Today's website systems are basically implemented according to the MVC design pattern. Therefore, using the characteristics of this design pattern can be used to check whether the user requirements are truly satisfied by analyzing the GUI(graphic user interface) of the view layer. The approach adopted in this paper is: we model according to the system's GUI page and software requirements document, and finally match and check the two models to check whether the product meets the functional requirements in the requirements document. This greatly reduces the amount of work that testers must do when performing software product acceptance.

CCS Concepts

• Software and its engineering → Software creation and management → Software verification and validation → Process validation → Acceptance testing.

Keywords

Automated test; Demand analysis; GUI; Model establishment;

1. INTRODUCTION

Software testing is an important stage of the software development process. Software testing costs up to 30% ~ 50% of the entire software development process [1]. As the front end of the software system, the appearance of the GUI of the graphical user interface brings great convenience to the users of the software system. It is the interface that the user interacts with the system. The function of the system is triggered by the user's operation on the graphical interface, and the result is also fed back to the user through it. [2] Therefore, for the entire software system, the rationality of the GUI is very important, and in order to ensure the rationality of the UI, testing is the most direct and effective means. [3] Therefore, we have researched and designed a system framework that can automatically test whether the product meets the customer's needs based on the GUI interface of the website system product, and design two corresponding models. The system framework is mainly composed of three parts, one is the configuration file layer, one is the engine execution layer, and the other is the model processing layer. The engine execution layer includes a GUI modeling engine and a structured document

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

CSAI '18, December 8–10, 2018, Shenzhen, China

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-6606-9/18/12...\$15.00

DOI: <https://doi.org/10.1145/3297156.3297215>

modeling engine for processing traditional user requirements. The GUI modeling method is the event flow graph model proposed by Memon et al. (2007) [4] Specifically, the GUI is decomposed into a hierarchy of modal conversations; this hierarchy is represented as an integration tree; Each modal conversation is represented as an event flow diagram that shows all possible event execution paths in the conversation; individual events are represented by their prerequisites and influences, that is the event coverage and the interaction between a test case of the UI software as a function coverage standard. In addition, there is a GUI test model based on event processing function proposed by Chen JC et al (2013) [5]. The prerequisite for modeling the detailed use cases in the software requirements document is that we need to agree to establish a standard description language (used to describe the event flow in the detailed use case). Then use the natural language processing tool to process the information contained in the event stream to build a use case model. The core of the article is to design the model and use the model to establish the connection between the requirements document and the GUI page.

2. DESIGN FRAMEWORK

The system is divided into three modules, one is the configuration file layer, one is the engine execution layer, and other is the model processing layer. Each layer is composed of its own components. The relationship framework between them is shown in Figure 1.

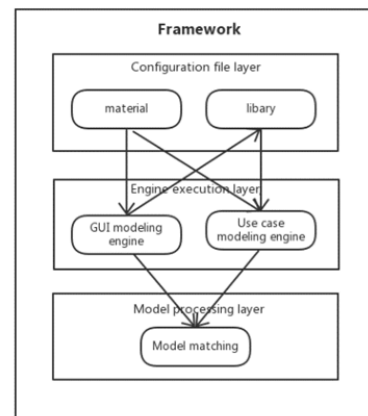


Figure 1. Frame architecture diagram.

The Configuration file layer contains a library of raw materials for the software requirements documentation and the GUI source code of the product being tested, as well as a library of pre-configured configuration files.

- The Engine actuating layer contains a GUI modeling engine and a use case modeling engine.

- The model execution layer contains a functional component for comparing and verifying two models (use case model and GUI model).
- The arrow indicates the direction of data flow.

3. DETAILED DESIGN

3.1 Defining the Expression Model

Actor	Actor's Goal	Use Case name
User roles	What the actor intends to accomplish	The naming system USES scenarios and numbering

Figure 2. Example of role target versus application example.

Use Case UC-#:	Name / Identifier [verb phrase]
Related Require-its:	List of the requirements that are addressed by this use case
Initiating Actor:	Actor who <i>initiates</i> interaction with the system to accomplish a <i>goal</i>
Actor's Goal:	Informal description of the initiating actor's goal
Participating Actors:	Actors that will <i>help</i> achieve the goal or <i>need to know</i> about the outcome
Preconditions:	What is <i>assumed</i> about the state of the system before the interaction starts
Postconditions:	What are the <i>results</i> after the goal is achieved or abandoned; i.e., what must be true about the system at the time the execution of this use case is completed
Flow of Events for Main Success Scenario:	
→ 1. The initiating actor delivers an <i>action</i> or <i>stimulus</i> to the system (the arrow indicates the direction of interaction, to- or from the system)	
← 2. The system's reaction or <i>response</i> to the stimulus; the system can also send a message to a participating actor, if any	
→ 3. ...	
Flow of Events for Extensions (Alternate Scenarios):	
What could go wrong? List the exceptions to the routine and describe how they are handled	
→ 1a. For example, actor enters invalid data	
← 2a. For example, power outage, network failure, or requested data unavailable	
...	
The arrows on the left indicate the direction of communication: → Actor's action; ← System's reaction	

Figure 3. Detailed use case description.

Identifier	Priority	Requirement
Number the requirements	Priority level of requirements. The higher the number, the higher the priority of the requirements	Describe what the user wants the system to do

Figure 4. User requirements table example.

Before proceeding with detailed module design, we must first select the constructed expression model; and define the specification of the requirements document use case. The first is to define the GUI model. The GUI model is a node based on a GUI component as a graph. Each GUI component raises one or more GUI events, and all components are related to each other through events that may occur. We make the following definition: the system has n numbers interactive model pages [4], which are denoted as $S = \{s_0, s_1, s_2, \dots, s_n\}$ ($n > 0$). Suppose that the initial modal interface [4] S_0 contains the $C = \{c_1, c_2, c_3, \dots, c_i\}$ ($i > 0$) page components and the n model interfaces [4] $S' = \{s'_1, s'_2, s'_3, \dots, s'_n\}$ at time t . The set of components in the model interface is denoted as $C' = \{c'_1, c'_2, c'_3, \dots, c'_i\}$. The event that the component may trigger is called $E = \{e_1, e_2, e_3, \dots, e_j\}$ ($0 \leq j$), which is recorded as $c_i = \{D, s_n / s'_n, E\}$; D represents the synonym of the component name or its name and the content contained in the component, s_n or s'_n represents the page in which the component exists, and E represents the set of next GUI events that the component may raise, such that The characteristics of the GUI program event-driven features can be fully expressed. On the modeling tool, we can learn from a tool called "GUI Ripper" developed by Memon et al (2007) [4], to automatically get event flow graph and integration tree. A detailed discussion of the tool is beyond the scope of this article; interested readers can refer to 35 articles previously published by Memon et al. In "GUI Ripper" combines the techniques of reverse engineering and algorithms to construct event flow graphs and integration trees. [2] Attached here is the event flow diagram and the link to the website of the integration

tree description, See the footer of this page for the URL information. A GUI component can be represented using an event flow diagram; intuitively, the event flow diagram represents all possible interactions between events in a GUI component, while the GUI component integrates the GUI interface of the entire system. However, the author also mentioned that this tool is not perfect, so it also needs human intervention to generate the event flow graph and the integration tree to ensure the accuracy of the model. After the GUI model is established, the requirements document is processed. There are three materials in the requirements document, which are especially important. It is the key to establish the demand use model. The role target pair application example (Figure 2), detailed use case table (Figure 3), as well as the user requirements table (Figure 4). Figure 2 through Figure 4 refer to the contents of "Software Engineering" by Ivan Marsic.

The idea of modeling the requirement use case is to first establish a demand model based on the relationship between the requirements and the user role and the use case, and then find the corresponding detailed use case table (Table 2) according to the use case name corresponding to the target of the user role in Table 1. In the detailed use case table, a use case model is established based on the described interaction flow between the system and the user. In the process, the following preconditions must be followed:

- The demand use case model builds models based on user roles, software requirements, use cases, behaviors, object components, and other attributes.
- The use case model is part of the requirements model
- The order of the demand use case model should be started from the verification of the basic requirements (the demand with a higher priority value), and the main use cases (the use cases without preconditions) are prioritized in the demand. After all the main use cases are completed, the relevant conditions are established. The use case model of the use case, which is a three-tiered tree structure with the subtree on the left taking precedence over the subtree on the right.
- The description language in the use case event stream must contain the complete subject-predicate object, the subject is the user role or system, the predicate is the subject-initiated event (such as clicking a button, filling out a form, etc.), and the object is the user-operated object (such as the GUI component).) or for the user, so the description of the event stream must contain the user or system (subject), event (predicate), object component (object).

According to the above definition, the demand model we build models based on user roles, software requirements, use cases, behaviors, and object components; respectively, which are represented as $A = \{a_1, a_2, a_3, \dots, a_n\}$, $R = \{r_1, r_2, r_3, \dots, r_n\}$, $U = \{u_1, u_2, u_3, \dots, u_n\}$, $B = \{b_1, b_2, b_3, \dots, b_n\}$, $O = \{o_1, o_2, o_3, \dots, o_n\}$. There are multiple use cases U_z ($0 < z \leq n$) under an a_i role, in which the user has multiple behaviors B_z ($0 < z \leq n$) in the main success or failure event stream of each use case, each b_i ($0 < i \leq z$) uses one or more O_z ($0 < z \leq n$), o_i needs to be determined according to the context and its role in other use cases, that is, to get the content and page it contains, and the set of events it may raise. A successful use case indicates that the user's goal is completed; the corresponding system requirement r_i is also completed when one or more user goals are reached.

3.2 Configuration Layer

The Configuration layer consists of two parts, a "material" for loading source files and requirements documents and

configuration files. The other component is used to collect events and page-related functions triggered by the user during execution of the engine, as shown in figure 5.

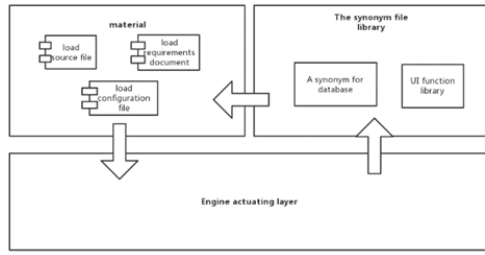


Figure 5. Configuration layer main components and libraries.

The reason for synonym inventory is because in natural language, there is a variety of vocabulary for describing a component; for example, in some countries, "addition" can be called "add" or just "+". The symbol indicates the case and the like; so it must be loaded into the configuration file in advance. Collecting events and their possible functions will help us better and better analyze the role of components and determine the model pages created by functions.

3.3 Engine Execution Layer

The engine execution layer contains the GUI modeling engine and the requirements document modeling engine. Their structure in the layer is shown in Figure 6.

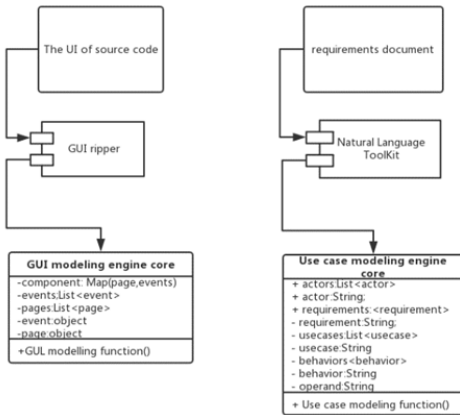


Figure 6. Engine execution layer main constructs and components.

The role of the GUI ripper is to analyze the source code of the UI interface during the test run, analyze the attributes we need, and then submit them to the core class to build the model. The use of natural language processing tools in the process of building a use-case model helps us to model the automation as much as possible, but in the process we need to check if the final result meets the requirements.

3.4 Model Processing Layer

The main job of the model processing layer is to match the two generation models. According to the requirements document use case model, there are multiple use cases under a certain role $a_i (0 < i \leq n)$, we find the event flow under one use case, and get the model of the event flow, the segment model contains multiple Object component O_z . In a user-initiated operation event, we find the component c_i in the corresponding GUI model according to the o_i (object component of the operation. The attributes of o_i and c_i are exactly the same ($o_i = \{D, s_i, E_z\} = c_i$), component name or

The content D is not necessarily consistent with the one displayed in the GUI in the event flow description, so the thesaurus needs to be loaded for checking to prevent errors or missing errors. If the entire stream O_z can find the path of the corresponding GUI component integration tree in the GUI integration tree model, it means that the use case is implemented in the page. And we verify that the requirement is implemented correctly, but also depends on the context of the object component. The above is the time t , all the page element set C_p after the operation behavior b_{i-n} occurs, the following is the t_i time ($t_i > t$), the entire element set C_z of the page after the behavior b_i occurs we can check the C_p and C_z elements. The relationship between the pages can be used to determine whether the requirements have been correctly implemented.

4. EXPERIMENT

The author of this paper participated in the development of the scientific research project management system project of the Hubei Provincial Health and Family Planning Commission. In the experiment, we tested the universal significance of the CAUD function to verify the feasibility of the study. We took a system requirement in the project and a real use case under a user role to experiment.

4.1 Requirements Document Section

Table 1 expresses a system requirement; three user roles need to have the ability to create subordinate accounts. The system administrator creates an organization recommendation unit account, and the organization recommendation unit continues to create the application unit account after obtaining the account. Based on the sequence principle established by the model, we prioritize the use case model without preconditions. Therefore, taking the goal of the system administrator: creating an organization recommendation unit account as an example, the implementation of the target corresponds to use case 6, as shown in Table 2. Table 3 shows the details of use case 6.

Table 1. System requirement

Identifier	priority	Requirement
REQ1	10	The system administrator, the organization department, the declaration department can create the subordinate organization account
.....		

Table 2 System administrator target and related use case table

Actor	Actor's Goal	Use Case Name
administrator	Create multiple organizational department accounts	UC-6
.....		

Table 3 Use Case 6 (UC-6) Detailed Use Case Table

Use Case UC-6:	Create multiple organizational department accounts
Related Require'ts:	REQ1
Initiating Actor:	administrator
Participating Actors:	

Preconditions:	
Postconditions:	After success, the department's account management page displays the new account and password created and the name of the organization
Flow of Events for Main Success Scenario:	
→ The administrator enters the username and password and clicks login	
← The system enters the administrator page	
→ The administrator clicks on the unit account label	
← System into the unit account management page	
→ The administrator clicks the add button	
← The system popup window of creates the unit account	
→ The user enters the company name and organization code and clicks the create button	
← The system returns to the account of department page, which contains the account table with the name, account number and password created just now	
Flow of Events for Extensions (Alternate Scenarios):	
.....	

Based on the above information, the demand model we built is shown in Figure 7. After the verification of the use case 6 is completed, the verification of the use case 7 can be performed, and after the use of the case 7 is verified, the verification of the use case 8 can be performed; Therefore, the left subtree is use case 6, the use case 7 is followed, and finally the use case 8.

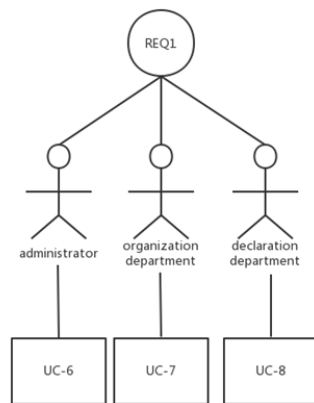


Figure 7. Requirement 1 model.

4.2 GUI Operation and Model Comparison

In order to illustrate the detailed process of use case 6, we intercepted the operated page in the real website and marked the components in the event with a red frame, as shown in Figure 8. We build the use case model based on the event flow of use case 6. At the same time, we find out that the GUI model that completed the build matches the completed use case model for the path and component, as shown in Figure 9.

In Figure 9, the same path of the use case model exists in the GUI model, from $c_1=\{\text{input}, s_0, \{e_2, e_3, e_4\}\}$ to $c_2=\{\text{input}, s_0, \{e_1, e_3, e_4\}\}$ to the other Component. Among them, e_1, e_2, e_3, e_4 , and e_4 correspond

to the input account, enter the password, click login and click to return to the four events in the real system (refer to the first screenshot in Figure 8). E_n indicates that the component may next have a collection of events. At time t_0 , click the add button component $c_5=\{\text{add}, s_2, E_3\}$, open the model page s'_2 in the s_2 page, and finally click the create button ($c_8=\{\text{create}, s'_2, E_4\}$) in the model page s'_2 , the system returns to the s_2 page, this component c_8 corresponds to the next event collection is E_4 , 'n' is all components in the s_2 page at time t_1 . Finally, comparing the difference between the component sets of page s_2 at t_0 and t_1 , it can be verified that a new set of organization recommendation unit name, account number and password component are added.

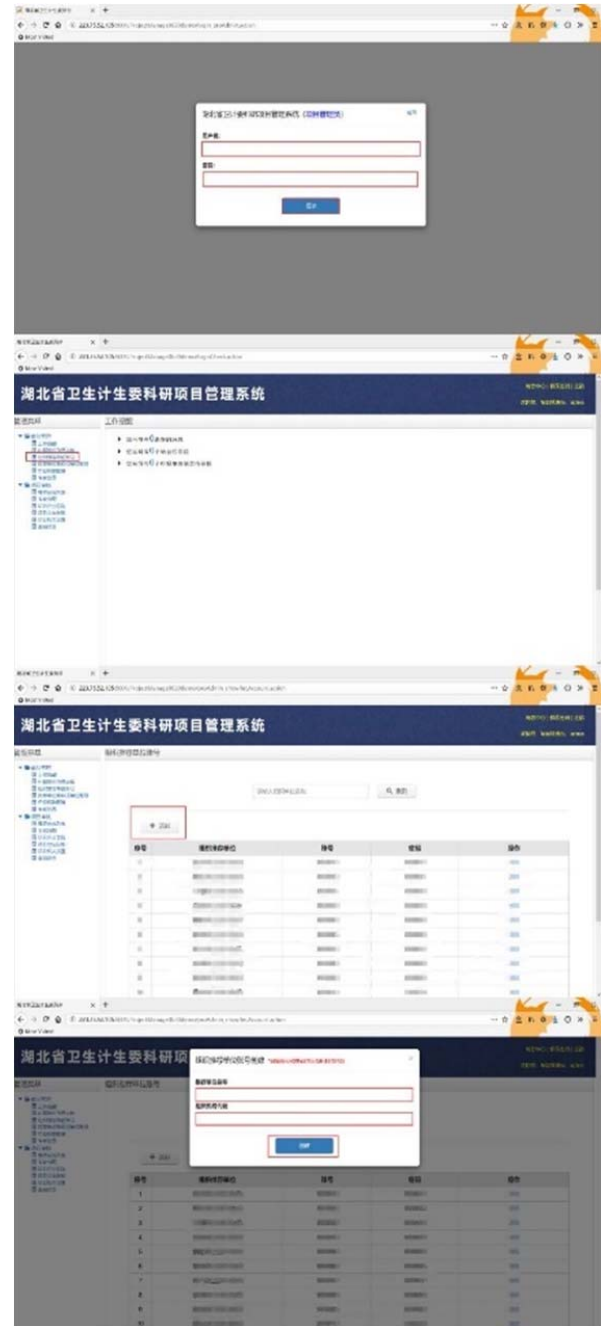


Figure 8. Website screenshot.

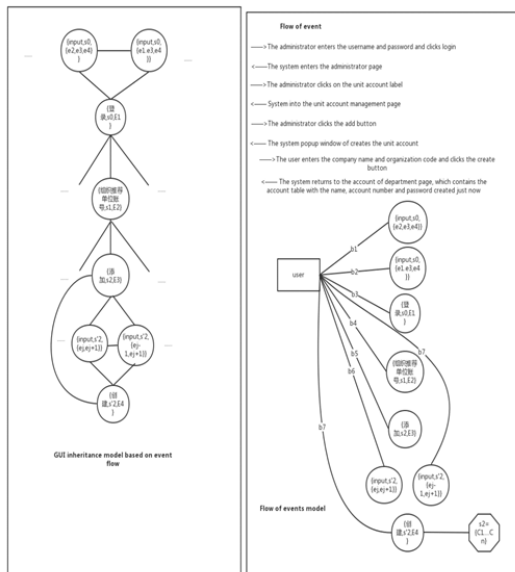


Figure 9. Model comparison.

5. CONCLUSION

There is a path of the use case model in the GUI model, so that it can be known that the interaction process of the use case 6 is implemented in the UI page, and then the page component and the component content of the S2 page are added according to different times t_0 and t_1 . It can be known that the use case is completed correctly. If all the use cases under REQ 1 are overwritten and completed correctly, then the requirement is correctly completed. Because the CAUD operations are highly similar, other operations

are not repeated here. It can be seen that the design framework can at least automatically check the user's simple requirements, thus helping to test the user's needs to a certain extent.

6. ACKNOWLEDGMENT

Our thanks to ACM SIGCHI for allowing us to modify templates they had developed.

7. REFERENCES

- [1] Beizer B. Software Testing Techniques. 2nd ed. Van Nostrand Reinhold Company Limited, 1990
- [2] MUB, ZHAN M K, HULF. Design and implementation of GUI automated testing framework based on XML [C] / / Proceedings of the 2009 WRI World Congress on Software Engineering. Washington, DC: IEEE Computer Society, 2009: 194 — 199.
- [3] GANOVSR, KILLMAR C, KHURSHIDS, et al. Test generation for graphical user interfaces based on symbolic execution [C] // Proceedings of the 3rd International Workshop on Automation of Software Test. New York: ACM Press, 2008: 33 — 40
- [4] Memon AM. An event-flow model of GUI-based applications for testing. Software Testing, Verification & Reliability, 2007,17(3): 137–157. [doi: 10.1002/stvr.364]
- [5] Chen JC, Xue YZ, Zhao C. Approach for GUI testing based on event handler function. Ruan Jian Xue Bao/ Journal of Software, 2013,24(12):2830 — 2842 (in Chinese).